



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА

«Разработка консольного приложения с применением ООП»

**ДИСЦИПЛИНА: «Архитектура автоматизированных систем
обработки информации и управления»**

Выполнил: студент гр. ИУ5-24Б

(Подпись) (Бондаренко Д.А.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю.Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Введение

Актуальность темы

В современных информационных системах управление данными является одной из ключевых задач. Для эффективной работы со структурированной информацией применяются системы управления базами данных и объектно-ориентированный подход. Консольные приложения на C# с SQLite позволяют создавать легковесные решения для учёта и анализа данных.

Цель работы:

Разработка консольного приложения на C# с применением ООП для управления данными о студентах и факультетах, а также формирование аналитических отчётов с использованием паттерна Fluent Interface.

Задачи:

1. Спроектировать классы-модели предметной области (вариант 4: Факультеты и Студенты).
2. Реализовать класс DatabaseManager для работы с SQLite.
3. Разработать интерактивное консольное меню.
4. Реализовать класс ReportBuilder с паттерном Fluent Interface.
5. Обеспечить три обязательных отчёта с JOIN, GROUP BY и агрегатными функциями.

Теоретическая часть

Предметная область

Согласно варианту 4 задания, справочная таблица — Факультеты, основная таблица — Студенты. Числовое поле — гра (средний балл успеваемости). Связь между таблицами — «один-ко-многим»: один факультет может содержать множество студентов.

Паттерн Fluent Interface

Fluent Interface — приём проектирования, при котором методы класса возвращают ссылку на текущий объект (this). Это позволяет выстраивать вызовы в цепочку, делая код более читаемым. Промежуточные методы (Query, Title, Header, ColumnWidths) настраивают отчёт, терминальные (Build, Print) выполняют запрос и выводят результат.

Расстояние Дамерау-Левенштейна

В основе алгоритма лежит метод динамического программирования Вагнера-Фишера с поправкой на транспозицию. Для строк длиной M и N создаётся матрица размером $(M+1) \times (N+1)$. Значение $D[i, j]$ — минимальное расстояние между префиксами. Рекуррентное соотношение учитывает операции вставки, удаления, замены и транспозиции.

Проектирование и реализация

Структура приложения

Приложение состоит из пяти основных компонентов:

1. Класс Faculty — модель справочной таблицы
2. Класс Student — модель основной таблицы с валидацией
3. Класс DatabaseManager — работа с SQLite
4. Класс ReportBuilder — формирование отчётов
5. Класс Program — меню и логика взаимодействия

Классы-модели

Класс Faculty

```
class Faculty
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Faculty(int id, string name)
    {
        Id = id;
        Name = name;
    }

    public Faculty() : this(0, "") { }

    public override string ToString()
    {
        return Id + ". " + Name;
    }
}
```

Класс содержит два свойства: Id (идентификатор) и Name (название факультета). Реализованы два конструктора: полный и по умолчанию (через цепочку this). Метод ToString переопределён для удобного вывода в формате «номер. название».

Класс Student

```

class Student
{
    public int Id { get; set; }
    public int FacultyId { get; set; }
    public string Name { get; set; }

    private double _gpa;
    public double Gpa
    {
        get => _gpa;
        set
        {
            if (value < 1 || value > 5)
                throw new ArgumentException("Балл должен быть от 1 до 5");
            gpa = value;
        }
    }

    public Student(int id, int facultyId, string name, double gpa) { ... }
    public Student() : this(0, 0, "", 0) { }

    public override string ToString()
    {
        return Id + "; " + Name + "; ID факультета: " + FacultyId + "; " + Gpa;
    }
}

```

Свойство Gpa имеет валидацию в set-аксессоре: значение должно быть от 1 до 5, иначе выбрасывается ArgumentException. Это гарантирует корректность данных на уровне модели. Свойство FacultyId является внешним ключом, связывающим студента с факультетом.

Класс DatabaseManager .Конструктор и инициализация

```

private string _connectionString;

public DatabaseManager(string dbPath)
{
    _connectionString = "Data Source=" + dbPath;
}

```

Строка подключения хранится как закрытое поле и используется во всех методах.

Создание таблиц

```
private void CreateTables()
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        CREATE TABLE IF NOT EXISTS faculty (
            faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
            faculty_name TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS student (
            student_id INTEGER PRIMARY KEY,
            faculty_id INTEGER NOT NULL,
            student_name TEXT NOT NULL,
            gpa REAL NOT NULL,
            FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
        );";
    cmd.ExecuteNonQuery();
}
```

Таблицы создаются с помощью CREATE TABLE IF NOT EXISTS, что гарантирует идемпотентность. Для faculty используется AUTOINCREMENT, для student — обычный PRIMARY KEY без автоинкремента (это нужно для последующей перенумерации). Внешний ключ связывает student.faculty_id с faculty.faculty_id.

Импорт CSV

```
private void ImportFacultiesFromCsv(string path)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    string[] lines = File.ReadAllLines(path, Encoding.UTF8);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 2) continue;
        var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO faculty (faculty_id, faculty_name)
VALUES (@id, @name)";
        cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
    }
}
```

```

        cmd.Parameters.AddWithValue("@name", parts[1]);
        cmd.ExecuteNonQuery();
    }
}

```

Файл читается с указанием Encoding.UTF8 для корректной обработки русских символов. Разделитель — точка с запятой. Первая строка пропускается (заголовки). Используются параметризованные запросы для защиты от SQL-инъекций.

CRUD-операции

Добавление студента:

```

public void AddStudent(Student student)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        INSERT INTO student (student_id, faculty_id, student_name, gpa)
        VALUES (@id, @facultyId, @name, @gpa)";
    student.Id = GetNextStudentId();
    cmd.Parameters.AddWithValue("@id", student.Id);
    cmd.Parameters.AddWithValue("@facultyId", student.FacultyId);
    cmd.Parameters.AddWithValue("@name", student.Name);
    cmd.Parameters.AddWithValue("@gpa", student.Gpa);
    cmd.ExecuteNonQuery();
}

```

ID нового студента вычисляется через метод GetNextStudentId(), который находит максимальный существующий ID и увеличивает его на 1.

Редактирование:

```

public void UpdateStudent(Student student)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        UPDATE student
        SET faculty_id = @facultyId, student_name = @name, gpa = @gpa
        WHERE student_id = @id";
    cmd.Parameters.AddWithValue("@id", student.Id);
    cmd.Parameters.AddWithValue("@facultyId", student.FacultyId);
}

```

```

cmd.Parameters.AddWithValue("@name", student.Name);
cmd.Parameters.AddWithValue("@gpa", student.Gpa);
cmd.ExecuteNonQuery();
}

```

Метод обновляет все поля записи по её ID.

Автонумерация после удаления

```

private void RenumberStudents()
{
    // 1. Сохраняем всех студентов в список
    var students = new List<(int facultyId, string name, double gpa)>();
    // ... SELECT faculty_id, student_name, gpa FROM student ORDER BY
student_id

    // 2. Очищаем таблицу
    // ... DELETE FROM student

    // 3. Вставляем заново с новыми ID по порядку
    for (int i = 0; i < students.Count; i++)
    {
        // ... INSERT INTO student VALUES (i + 1, ...)
    }
}

```

Алгоритм работает в три шага: сохраняет все записи в память, удаляет всё из таблицы, вставляет заново с ID от 1 до N. Это гарантирует, что после удаления нумерация остаётся непрерывной.

Удаление:

```

public void DeleteStudent(int id)
{
    // ... DELETE FROM student WHERE student_id = @id
    RenumberStudents();
}

```

После выполнения DELETE вызывается RenumberStudents для обновления нумерации.

Выполнение произвольных запросов

```
public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
{
    // ... выполняет SQL, возвращает имена столбцов и строки данных
}
```

Метод возвращает кортеж: массив названий колонок и список строк. Используется классом ReportBuilder для получения данных отчётов.

Класс ReportBuilder (Fluent Interface)

```
class ReportBuilder
{
    private DatabaseManager _db;
    private string _sql = "";
    private string _title = "";
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();

    public ReportBuilder(DatabaseManager db) { _db = db; }

    public ReportBuilder Query(string sql) { _sql = sql; return this; }
    public ReportBuilder Title(string title) { _title = title; return this; }
    public ReportBuilder Header(params string[] columns) { _headers = columns;
return this; }
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths;
return this; }

    public string Build() { ... }
    public void Print() { Console.WriteLine(Build()); }
}
```

Каждый промежуточный метод (Query, Title, Header, ColumnWidths) сохраняет переданное значение во внутреннем поле и возвращает this. Это позволяет выстраивать цепочку вызовов. Терминальный метод Build выполняет запрос через _db.ExecuteQuery, форматирует результат через StringBuilder с выравниванием колонок. Print вызывает Build и выводит результат в консоль.

Пример использования:

```
new ReportBuilder(db)
    .Query("SELECT ...")
    .Title("Список студентов по факультетам")
    .Header("Имя студента", "Факультет", "Балл")
    .ColumnWidths(25, 30, 10)
    .Print();
```


Консольное меню

Главный цикл программы реализован через do-while:

```
do
{
    Console.WriteLine("Управление:");
    Console.WriteLine("1. Показать все факультеты");
    Console.WriteLine("2. Показать всех студентов");
    Console.WriteLine("3. Добавить студента");
    Console.WriteLine("4. Редактировать студента");
    Console.WriteLine("5. Удалить студента");
    Console.WriteLine("6. Отчеты");
    Console.WriteLine("0. Выход");

    choice = Console.ReadLine();

    try
    {
        switch (choice)
        {
            case "1": ShowFaculties(db); break;
            case "2": ShowStudents(db); break;
            case "3": AddStudent(db); break;
            case "4": EditStudent(db); break;
            case "5": DeleteStudent(db); break;
            case "6": ReportsMenu(db); break;
            case "0": Console.WriteLine("До свидания!"); break;
            default: Console.WriteLine("Неверный пункт."); break;
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
    }
} while (choice != "0");
```

Весь ввод обрабатывается через try-catch, что предотвращает падение программы при некорректных действиях пользователя.

Проверки при вводе данных

При добавлении студента проверяется существование факультета:

```
bool facultyExists = false;
foreach (var f in faculties)
{
```

```

    if (f.Id == facultyId)
    {
        facultyExists = true;
        break;
    }
}
if (!facultyExists)
{
    Console.WriteLine("Ошибка: факультет с ID " + facultyId + " не найден.");
    return;
}

```

Для ввода чисел используется TryParse с CultureInfo.InvariantCulture, что позволяет корректно обрабатывать как точку, так и запятую в десятичных числах независимо от системной локали.

При редактировании пустой ввод означает «оставить без изменений»:

```

Console.Write("Новое имя (" + student.Name + "): ");
string input = Console.ReadLine();
if (!string.IsNullOrEmpty(input))
    student.Name = input.Trim();

```

Три обязательных отчёта

Отчёт 1 — JOIN

Список студентов с названиями факультетов:

```

new ReportBuilder(db)
    .Query(@"
        SELECT s.student_name, f.faculty_name, s.gpa
        FROM student s
        JOIN faculty f ON s.faculty_id = f.faculty_id
        ORDER BY s.student_id")
    .Title("Список студентов по факультетам")
    .Header("Имя студента", "Факультет", "Балл")
    .ColumnWidths(25, 30, 10)
    .Print();

```

Используется INNER JOIN для соединения таблиц по внешнему ключу. Результат сортируется по ID студента.

Отчёт 2 — GROUP BY + COUNT

Количество студентов на каждом факультете:

```

new ReportBuilder(db)
    .Query(@"
        SELECT f.faculty_name, COUNT(*) AS students_count

```

```

FROM student s
JOIN faculty f ON s.faculty_id = f.faculty_id
GROUP BY f.faculty_name
ORDER BY f.faculty_name")
.Title("Количество студентов на факультетах")
.Header("Факультет", "Кол-во студентов")
.ColumnWidths(35, 20)
.Print();

```

Агрегатная функция COUNT(*) подсчитывает количество записей в каждой группе. Группировка по названию факультета, сортировка по алфавиту.

Отчёт 3 — GROUP BY + AVG

Средний балл по факультетам:

```

new ReportBuilder(db)
.Query(@"
SELECT f.faculty_name, ROUND(AVG(s.gpa), 2) AS avg_gpa
FROM student s
JOIN faculty f ON s.faculty_id = f.faculty_id
GROUP BY f.faculty_name
ORDER BY avg_gpa DESC")
.Title("Средний балл по факультетам")
.Header("Факультет", "Средний балл")
.ColumnWidths(35, 15)
.Print();

```

AVG вычисляет среднее арифметическое, ROUND округляет до двух знаков. Сортировка по убыванию среднего балла.

Итоги:

В рамках выполнения домашнего задания было разработано и протестировано консольное приложение, реализующее управление данными о студентах и факультетах. Программа полностью соответствует функциональным требованиям:

- Хранит данные в SQLite
- Позволяет добавлять, редактировать и удалять записи через консольное меню
- Формирует три аналитических отчёта с помощью паттерна Fluent Interface
- Обеспечивает валидацию вводимых данных на уровне модели
- Поддерживает автонумерацию после удаления записей

Выводы по теоретической части:

В ходе работы были изучены принципы объектно-ориентированного программирования на C#, применение паттерна Fluent Interface для построения отчётов, работа с SQLite через Microsoft.Data.Sqlite. Освоены методы динамического программирования на примере алгоритма Вагнера-Фишера.

Возможные пути улучшения:

- Ослабление связи между ReportBuilder и DatabaseManager через интерфейсы
- Добавление экспорта отчётов в CSV и текстовые файлы
- Реализация поиска по имени студента
- Оптимизация автонумерации для больших объёмов данных